

Course: Hands-On Data Analytics with Python

Eli Zamir

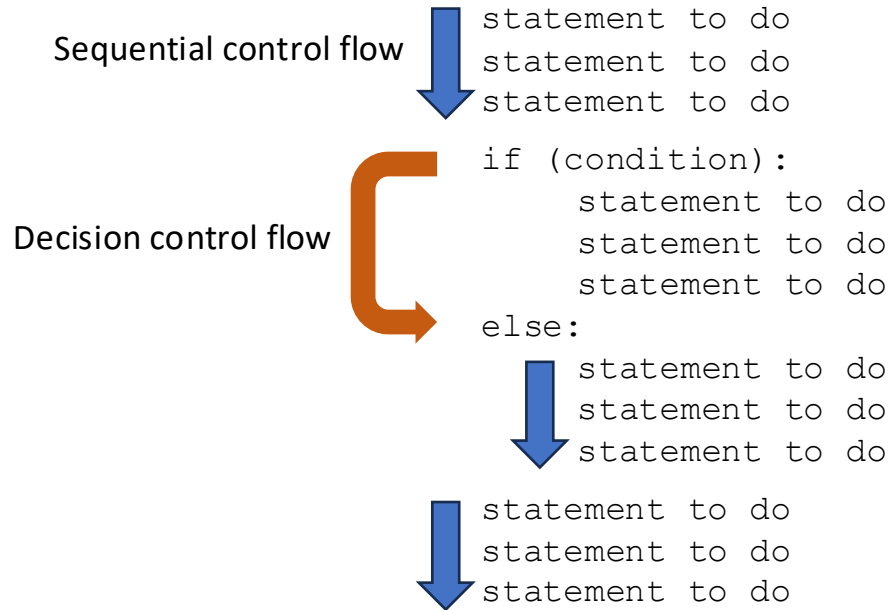
Session 3: Core Python programming (part 2)

Control flows

Control flows determine the order of instructions execution, and which instructions are executed, in a program.

For example:

- Sequential control flow: Instructions are executed in the order that they are written (the default flow)
- Decision control flow: The program chooses different paths based on conditions



Control flows in Python

- Sequential control flow: Instructions are executed in the order in which they are written
 - This is the default flow
- Decision control flow: The program chooses different paths based on conditions
 - `if, elif, else`
 - `match, case`
- Iteration (loop) control flow: Repeatedly executes a block of code according to a given scheme or rule
 - `for...in...`
 - `while`
- Jump control flow: Change the normal flow inside a loop
 - `break`
 - `continue`
- Function calls control flow: Control jumps to a function, runs its body, then returns to the caller
 - `return`
 - `(yield)`
- Error control flow: Changes execution path when an error or exceptional condition occurs
 - `try, except, else, finally,`
 - `raise, assert`

Control flows in Python

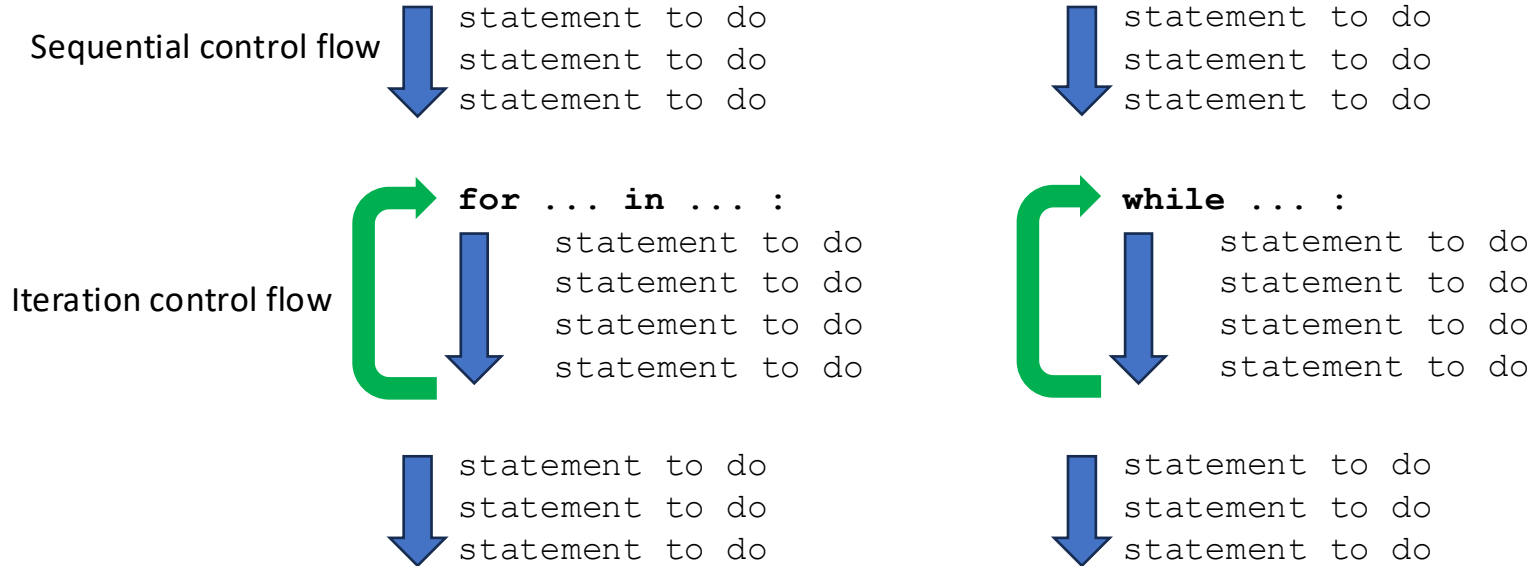
- Sequential control flow: Instructions are executed in the order in which they are written
 - This is the default flow
- Decision control flow: The program chooses different paths based on conditions
 - `if, elif, else`
 - `match, case`
- Iteration (loop) control flow: Repeatedly executes a block of code according to a given scheme or rule
 - `for...in...`
 - `while`
- Jump control flow: Change the normal flow inside a loop
 - `break`
 - `continue`
- Function calls control flow: Control jumps to a function, runs its body, then returns to the caller
 - `return`
 - `(yield)`
- Error control flow: Changes execution path when an error or exceptional condition occurs
 - `try, except, else, finally,`
 - `raise, assert`

Iteration (loop) control flow

Iteration (looping) control flow:

Repeatedly executes a block of code according to a given scheme or rule

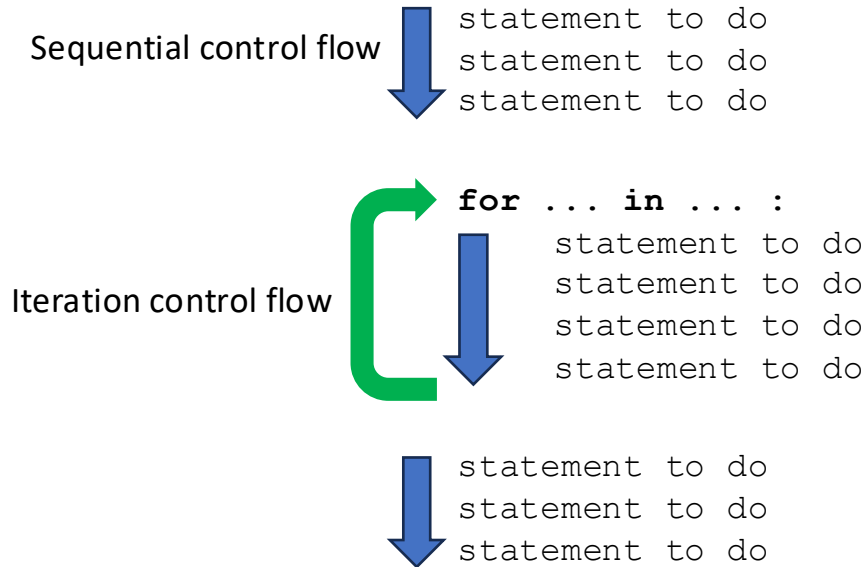
Two types of loops: `for` and `while`



for iterations

for iterations:

Repeatedly executes a block of code over a defined range or sequence of items.



for loop: iterating through a range object

for iterations can be performed directly on a range object

```
range(10)
```

```
range(0, 10)
```

```
type(range(10))
```

```
range
```

```
for x in range(10):  
    print(x)
```

0
1
2
3
4
5
6
7
8
9

```
for x in range(200,0,-20):  
    print(x)
```

200
180
160
140
120
100
80
60
40
20

```
for x in range(1,10,1):  
    y = x**2  
    print(f"{x}^2 = {y}")
```

1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81

for loop: iterating through list, tuple and dict objects

for iterations can also be applied on any iterable data structure, in the order provided by that structure.

iterating through a list

```
letters_list = ["a", "b", "c", "d", "e"]

for x in letters_list:
    print(x, end=" ")

a b c d e
```

iterating through a tuple

```
letters_tuple = ("a", "b", "c", "d", "e")

for x in letters_tuple:
    print(x, end=" ")

a b c d e
```

iterating through a set

```
letters_set = {"a", "b", "c", "d", "e"}

for x in letters_set:
    print(x, end=" ")

c d a b e
```

Order is not preserved !



iterating through a dict

```
letters_dictionary = {"a":1,"b":2,"c":3,"d":4,"e":5}

for key in letters_dictionary:
    print(f"{key}:{letters_dictionary[key]}", end=" ")

a1 b2 c3 d4 e5
```


Iterating “in parallel” through multiple corresponding lists

How to iterate through multiple corresponding lists?

For example:

```
planets_names = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"]
planets_distance = [0.39, 0.72, 1.00, 1.52, 5.20, 9.58, 19.2, 30.1]
```

Iterating via an explicit index is possible, but not recommended:

```
for index in range(len(planets_names)):
    print(f"The mean distance of {planets_names[index]} from the sun is: {planets_distance[index]:2f} AU")
```

The mean distance of	Mercury	from the sun is:	0.39 AU
The mean distance of	Venus	from the sun is:	0.72 AU
The mean distance of	Earth	from the sun is:	1.00 AU
The mean distance of	Mars	from the sun is:	1.52 AU
The mean distance of	Jupiter	from the sun is:	5.20 AU
The mean distance of	Saturn	from the sun is:	9.58 AU
The mean distance of	Uranus	from the sun is:	19.20 AU
The mean distance of	Neptune	from the sun is:	30.10 AU

Iterating “in parallel” through multiple corresponding lists via `zip()`

The recommended approach: using the `zip` function

```
planets_names = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"]
planets_distance = [0.39, 0.72, 1.00, 1.52, 5.20, 9.58, 19.2, 30.1]
```



```
for planet_name, planet_distance in zip(planets_names, planets_distance):
    print(f"The mean distance of {planet_name} \t from the sun is {planet_distance:.2f} AU")
```

```
The mean distance of Mercury from the sun is 0.39 AU
The mean distance of Venus from the sun is 0.72 AU
The mean distance of Earth from the sun is 1.00 AU
The mean distance of Mars from the sun is 1.52 AU
The mean distance of Jupiter from the sun is 5.20 AU
The mean distance of Saturn from the sun is 9.58 AU
The mean distance of Uranus from the sun is 19.20 AU
The mean distance of Neptune from the sun is 30.10 AU
```

Advantages:

- No need to explicitly deal with an index and no need to use `range(len())`
- Safer in case lists are not of the same size
- Cleaner syntax

Obtaining enumerating index in `for` iterations using `enumerate()`

Sometimes we do need to have an access to the iteration index

The recommended way to achieve this is via the `enumerate()` function

```
letters_list = ["a", "b", "c", "d", "e"]  
  
for index, letter in enumerate(letters_list):  
    print(index, letter)
```

```
0 a  
1 b  
2 c  
3 d  
4 e
```

for iterations combining enumerate() with zip()

Using enumerate() in combination with zip():

```
planets_names = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"]
planets_distance = [0.39, 0.72, 1.00, 1.52, 5.20, 9.58, 19.2, 30.1]

print(f"index\tplanet_name\tplanet_name")

for index, (planet_name, planet_distance) in enumerate(zip(planets_names, planets_distance)):
    print(f"{index}\t{planet_name}\t\t{planet_distance:.2f}")
```

index	planet_name	planet_name
0	Mercury	0.39
1	Venus	0.72
2	Earth	1.00
3	Mars	1.52
4	Jupiter	5.20
5	Saturn	9.58
6	Uranus	19.20
7	Neptune	30.10

Nested loops

External loop

Internal loop

```
items_list = ["chair", "pen", "car"]
colors_list = ["brown", "blue", "black", "white"]

for item in items_list:
    print(f"{item.upper()}:")

    for color in colors_list:
        print(f"This is a {color} {item}.")

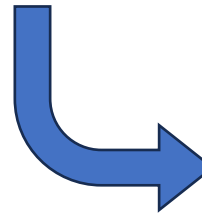
    print("-----")
```

Output

```
CHAIR:
This is a brown chair.
This is a blue chair.
This is a black chair.
This is a white chair.
-----
```

```
PEN:
This is a brown pen.
This is a blue pen.
This is a black pen.
This is a white pen.
-----
```

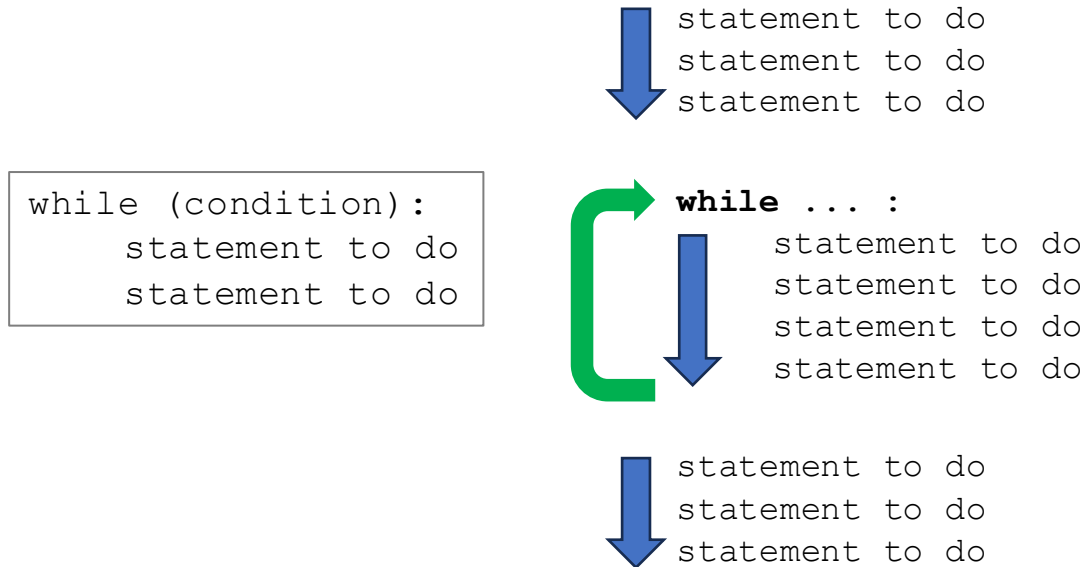
```
CAR:
This is a brown car.
This is a blue car.
This is a black car.
This is a white car.
-----
```



Control Flow: `while` loops

`while` iteration:

Repeatedly executes a block of code as long as a specified condition remains true.



while loop (simple example)

```
while (condition):  
    statement to do  
    statement to do
```

```
total_tries = 5  
current_try = 0  
  
while current_try < total_tries:  
    current_try = current_try + 1  
    print(f"Try number: {current_try}")  
print("No more tries !")
```

```
Try number: 1  
Try number: 2  
Try number: 3  
Try number: 4  
Try number: 5  
No more tries !
```

Control flows in Python

- Sequential control flow: Instructions are executed in the order in which they are written
 - This is the default flow
- Decision control flow: The program chooses different paths based on conditions
 - `if, elif, else`
 - `match, case`
- Iteration (loop) control flow: Repeatedly executes a block of code according to a given scheme or rule
 - `for...in...`
 - `while`
- Jump control flow: Change the normal flow inside a loop
 - `break`
 - `continue`
- Function calls control flow: Control jumps to a function, runs its body, then returns to the caller
 - `return`
 - `(yield)`
- Error control flow: Changes execution path when an error or exceptional condition occurs
 - `try, except, else, finally,`
 - `raise, assert`

“continue” and “break” in a for loop

“continue”: skips the current iteration

```
for x in range(1,11):  
    if x % 2 == 0:  
        continue  
    print(x)
```



1
3
5
7
9

“break”: exits the iteration loop

```
for x in range(1,11):  
    if x % 2 == 0:  
        print(x)  
        break
```



2

“continue” and “break” in a while loop

“continue”: skip the current iteration

```
current_try = 0

while current_try < 5:
    current_try = current_try + 1
    if current_try == 3:
        continue
    print(f"Try number: {current_try}")
print("No more tries !")
```

```
Try number: 1
Try number: 2
Try number: 4
Try number: 5
No more tries !
```

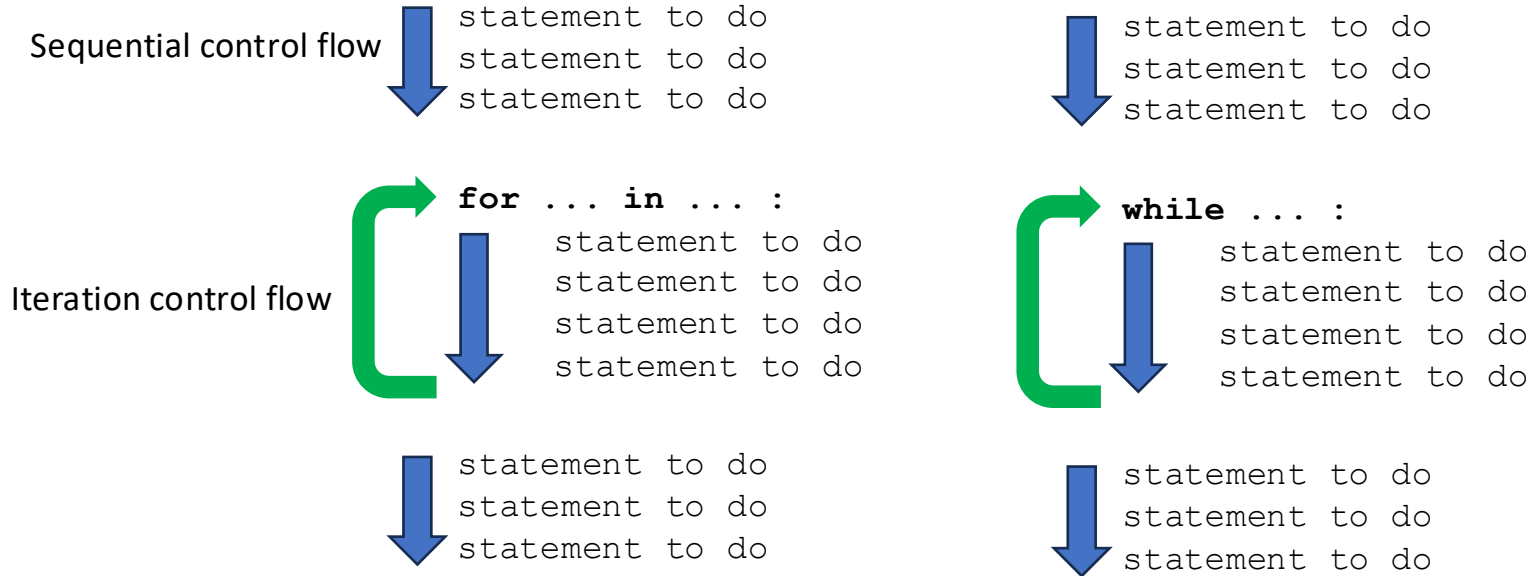
“break”: exit the iteration loop

```
current_try = 0

while current_try < 5:
    current_try = current_try + 1
    if current_try == 3:
        break
    print(f"Try number: {current_try}")
print("No more tries !")
```

```
Try number: 1
Try number: 2
No more tries !
```

Choosing between `for` loop and `while` loop



`for` loop: If we need to iterate over a given list (/set) of elements

`while` loop: If we need to iterate until a given condition becomes False

List comprehension

List comprehension:

Enables to generate a list based on an input iterable data structure by applying a given operation, $f(x)$, on each of the items

Syntax: `[f(x) for x in iterable if condition]`

```
[x for x in range(1,31) if x%2 == 0]
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30]
```

```
[x**2 for x in range(1,31) if x%2 == 0]
```

```
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400, 484, 576, 676, 784, 900]
```

```
mixed_list = ["a", 5.2, True, 4, (3,4), 1+1j, {2,3,4}, [1,2,3,4]]  
[type(b) for b in mixed_list]
```

```
[str, float, bool, int, tuple, complex, set, list]
```

Control flows in Python

- Sequential control flow: Instructions are executed in the order in which they are written
 - This is the default flow
- Decision control flow: The program chooses different paths based on conditions
 - `if, elif, else`
 - `match, case`
- Iteration (loop) control flow: Repeatedly executes a block of code according to a given scheme or rule
 - `for...in...`
 - `while`
- Jump control flow: Change the normal flow inside a loop
 - `break`
 - `continue`
- Function calls control flow: Control jumps to a function, runs its body, then returns to the caller
 - `return`
 - `(yield)`
- Error control flow: Changes execution path when an error or exceptional condition occurs
 - `try, except, else, finally,`
 - `raise, assert`

Toward functions

For a given rectangle (67 cm width, 8 cm length), we want to calculate:

- the area
- the perimeter
- the aspect ratio
- the diagonal length

```
width = 67; length = 8

area = width * length
primeter = 2*width + 2*length
aspect_ratio = width / length
diagonal = (width**2 + length**2)**0.5

print(f"{area}\t{primeter}\t{aspect_ratio:.2f}\t{diagonal:.2f}")
```

536	150	8.38	67.48
-----	-----	------	-------

Next, we want to calculate the same, but for another rectangle (14 cm width, 10 cm length):

→ requires to repeat the same code (beside the different input)

```
width = 14; length = 10

area = width * length
primeter = 2*width + 2*length
aspect_ratio = width / length
diagonal = (width**2 + length**2)**0.5

print(f"{area}\t{primeter}\t{aspect_ratio:.2f}\t{diagonal:.2f}")
```

140	48	1.40	17.20
-----	----	------	-------

Functions

function:

- A named block of code that performs a certain set of tasks
- Can accept input values and return results
- Enables reusability and modular organization of the code

def command to define a function

return () returns the values of the variables which are stated within the () and ends the function's execution

Syntax
(without returned values)

```
def function_name_a():  
    # code line 1  
    # code line 2  
    # ...
```

Input parameters

Syntax
(with returned values)

```
def function_name_b():  
    # code line 1  
    # code line 2  
    # ...  
    return()
```

Returned value(s)

Functions - a simple example

Defining a function “analyze_rect”
to perform this computation:

```
def analyze_rect(width, length):  
  
    area = width * length  
    primeter = 2*width + 2*length  
    aspect_ratio = width / length  
    diagonal = (width**2 + length**2)**0.5  
  
    print(f"{area}\t{primeter}\t{aspect_ratio:.2f}\t{diagonal:.2f}")
```

Calling the function enable to re-use
the same code on other input values
(here, another rectangle)

analyze_rect(67, 8)			
536	150	8.38	67.48
analyze_rect(14, 10)			
140	48	1.40	17.20

Functions - a simple example

```
def analyze_rect(width, length):  
  
    area = width * length  
    primeter = 2*width + 2*length  
    aspect_ratio = width / length  
    diagonal = (width**2 + length**2)**0.5  
  
    print(f"{area}\t{primeter}\t{aspect_ratio:.2f}\t{diagonal:.2f}")
```

We can use a function within a loop:

```
for width in [5, 6, 7]:  
    for length in [3,4,5]:  
        analyze_rect(width, length)
```

15	16	1.67	5.83
20	18	1.25	6.40
25	20	1.00	7.07
18	18	2.00	6.71
24	20	1.50	7.21
30	22	1.20	7.81
21	20	2.33	7.62
28	22	1.75	8.06
35	24	1.40	8.60

Functions - a simple example

Adding to the function `return`, to return results:

```
def analyze_rect(width, length):  
  
    area = width * length  
    perimeter = 2*width + 2*length  
    aspect_ratio = width / length  
    diagonal = (width**2 + length**2)**0.5  
  
    return(area, perimeter, aspect_ratio, diagonal)
```

```
results = analyze_rect(14, 10)
```

The default return of multiple values is a tuple:

```
results
```

```
(140, 48, 1.4, 17.204650534085253)
```

```
type(results)
```

```
tuple
```

Different ways to retrieve the results:

```
a, b, c, d = analyze_rect(14, 10)  
print(a, b, c, d)
```

```
140 48 1.4 17.204650534085253
```

A function can be called within a call to another function:

```
min(analyze_rect(14, 10))
```

```
1.4
```

Using a function within another function

Before a function is called, all the functions it calls must be already defined !

```
def display_price(price, currency):  
    print(f"The price is {price} {currency}.")  
  
def calculate_payments(price, n_payments):  
    return (price / n_payments)  
  
def make_offer(price, currency, n_payments):  
    display_price(price, currency)  
    payment = calculate_payments(price, n_payments)  
    print(f"It can be paid by {n_payments} payments of {payment} {currency} each.")  
  
make_offer(500, "Euro", 10)
```

The price is 500 Euro.
It can be paid by 10 payments of 50.0 Euro each.

The function input arguments: Positional arguments

Positional arguments:

- The arguments that are passed to a function in a specific order
- The position of each argument matters, as it gets assigned to the corresponding parameter based on the order in the function definition

```
def myfunction1(width, length):  
    print(width, length, sep=", ")
```

```
myfunction(10,20)
```

```
10, 20
```

```
myfunction(20,10)
```

```
20, 10
```

If expected input is not provided → Error

```
myfunction1()
```

```
TypeError
```

```
Traceback (most recent call last)
```

```
Cell In[77], line 1
```

```
----> 1 myfunction1()
```

```
TypeError: myfunction1() missing 2 required positional arguments: 'width' and 'length'
```

```
myfunction1(10)
```

```
TypeError
```

```
Traceback (most recent call last)
```

```
Cell In[78], line 1
```

```
----> 1 myfunction1(10)
```

```
TypeError: myfunction1() missing 1 required positional argument: 'length'
```

The function input arguments: Keyword arguments

Keyword arguments:

- The arguments that are passed to a function using the parameter name
- The parameter name is used to specify which value goes into which variable

```
def myfunction1(width, length):  
    print(width, length, sep=", ")
```

Input as positional arguments

```
myfunction1(7, 35)
```

7, 35

```
myfunction1(35, 7)
```

35, 7

Input as keyword arguments

```
myfunction1(width=7, length=35)
```

7, 35

```
myfunction1(length=35, width=7)
```

7, 35

The function input arguments: default values

Setting default values for width and length

```
def myfunction2(width = 8, length = 27):  
    print(width, length, sep=", ")
```

If values are not provided, the defaults are used

```
myfunction2()
```

```
8, 27
```

If values are given, they "overwrite" the defaults

```
myfunction2(30, 40)
```

```
30, 40
```

Unpacking an arbitrary number of positional arguments with *args

*: Unpacking *args (positional arguments)

- Receives extra positional arguments and packs them in one tuple
- Used when the number of positional arguments to be passed to a function is not known ahead
- Enables to accept a variable number of positional arguments

```
def select_sort_even_numbers(*args):  
    even_numbers = [number for number in args if number % 2 == 0]  
    even_numbers.sort()  
    print("Even numbers:", even_numbers)
```

```
select_sort_even_numbers(10, 2, 3, 4, 7, 11)
```

```
Even numbers: [2, 4, 10]
```

```
select_sort_even_numbers(127, 2096, 4029, 13, 14)
```

```
Even numbers: [14, 2096]
```

Unpacking an arbitrary number of keyword arguments with `**kwargs`

`**`: Unpacking `**kwargs` (keyword arguments)

- Used to handle an arbitrary number of keyword arguments
- Collects all the remaining keyword arguments into a dictionary (`dict`)
- Enables to accept a variable number of keyword arguments

```
def print_details(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")
```

```
print_details(animal="Dog", age=7, weight=5)
```

```
animal: Dog  
age: 7  
weight: 5
```

```
print_details(animal="Cat", weight=2, color="Black", fur="Short")
```

```
animal: Cat  
weight: 2  
color: Black  
fur: Short
```

```
print_details(product="Car", price="15000 Euro")
```

```
product: Car  
price: 15000 Euro
```


Rules for the order of input arguments of mixed types with `*args` and `**kwargs`

Input arguments in functions must appear in the following order (from left to right):

1. Positional-only arguments
2. Positional or keyword arguments
3. `*args` (variable positional arguments)
4. Keyword-only arguments
5. `**kwargs` (variable keyword arguments)

Additional rules:

- All parameters before “/” must be passed positionally
- All parameters after the “*” must be passed using keywords
- All required parameters without defaults must come before parameters with defaults

A (complicated) example:

```
def mixed_arguments(a, b, /, c, *args, d=10, **kwargs):
```

Providing input arguments of mixed types with *arg and ** kwargs (example)

```
def mixed_arguments(a, b, /, c, *args, d=10, **kwargs):  
    print(f"a={a}, b={b}, c={c}")  
    print(f"args={args}")  
    print(f"d={d}")  
    print(f"kwargs={kwargs}")
```

```
mixed_arguments(1,2,3,4,5,6,7,animal="Dog", weight="6", color="Brown")
```

```
a=1, b=2, c=3
```

```
args=(4, 5, 6, 7)
```

```
d=10
```

```
kwargs={'animal': 'Dog', 'weight': '6', 'color': 'Brown'}
```

Functions and variable scope: local variables

Local variable: A variable created inside a function

The scope of a local variable is limited to the function in which it got defined. It is not accessible outside this function.

```
def add_two(x):  
    y = x + 2  
    return(y)
```

```
add_two(5)
```

```
7
```

```
y
```

```
-----  
NameError  
Cell In[10], line 1  
----> 1 y
```

```
NameError: name 'y' is not defined
```

```
x
```

```
-----  
NameError  
Cell In[11], line 1  
----> 1 x
```

```
NameError: name 'x' is not defined
```

Functions and variable scope: global variables

Global variable:

- A variable created outside a function is a global variable
- Functions can read global variables directly

```
number = 23

def add_a_number(x):
    y = x + number
    return(y)

add_a_number(2)
```

25

! Still, the right practice is to provide the function all the variables it needs as input arguments

Global variables within functions

To modify a global variable inside a function “`global`” should be used.

```
number = 23

def add_a_number(x):
    global y
    y = x + number
    return(y)

add_a_number(2)
```

25

y

25

```
number = 23

def modify_number():
    number = 50

modify_number()
number
```

23

```
number = 23

def modify_number():
    global number
    number = 50

modify_number()
number
```

50

- ! As a rule: better to avoid affecting or defining global variables within function, because this:
- hampers modularity
 - is prone to errors

Mutability of objects in Python

Variables are names (labels) that reference objects in memory

Mutability: whether an object can be changed after it is created

Immutable objects cannot be modified.

➤ Therefore, any “change” in it actually creates a new object.

Immutable objects:

- int
- float
- str
- tuple

Mutable objects can be modified in place.

➤ Therefore, changes in such an object affect all the references to it. (!!!)

Exmplr for mutable objects:

- list
- dict
- set

Mutability of objects in Python

Immutable object (e.g. `int`)

```
a = 10
b = a      # b refers to the same value as a

b = b + 1  # create a NEW integer and reassign b

print(f"a: {a}")
print(f"b: {b}")

a: 10
b: 11
```

Mutable object (e.g. `list`)

```
a = [1, 2]
b = a      # b refers to the SAME list as a

b.append(3) # modify the shared list

print(f"a: {a}")
print(f"b: {b}")

a: [1, 2, 3]
b: [1, 2, 3] ← Also changed! (not good)
```

```
a = [1, 2]
b = a.copy() # make a NEW, separate list with
              # the values of a

b.append(3)  # modify list b in place

print(f"a: {a}")
print(f"b: {b}")

a: [1, 2]
b: [1, 2, 3] 
```

The importance of mutability for functions

Variables are names (labels) that reference objects in memory

Python uses call-by-object-reference

The function parameter becomes a new name that refers to the same object passed in.

Immutable object (e.g. int)

```
def add_one(x):  
    x = x + 1  
    print(f"Inside: x = {x}")  
  
a = 5  
print(f"Outside, before call: a = {a}")  
  
add_one(a)  
  
print(f"Outside, after call: a = {a}")
```

Outside, before call: a = 5  a unaffected
Inside: x = 6
Outside, after call: a = 5 

Mutable object (e.g. list)

```
def add_item(x):  
    x.append(3)  
    print(f"Inside: x = , {x}")  
  
a = [1, 2]  
print(f"Outside, before call: a = {a}")  
  
add_item(a)  
print(f"Outside, after call: a = {a}")
```

Outside, before call: a = [1, 2]  a affected
Inside: x = , [1, 2, 3]
Outside, after call: a = [1, 2, 3]  (not good)

The safe way to work with mutable input variables in functions

Making a separate list,
with the content of x



```
def add_item(x):  
    x = x.copy()  
    x.append(3)  
    print(f"Inside: x = , {x}")  
  
a = [1, 2]  
print(f"Outside, before call: a = {a}")  
  
add_item(a)  
print(f"Outside, after call: a = {a}")
```

```
Outside, before call: a = [1, 2] ←  
Inside: x = , [1, 2, 3]  
Outside, after call: a = [1, 2] ←
```

Now a is unaffected

`x = x.copy()` creates a new object and rebinds the local name x.

Classes and Objects

- An "object": a combination of features and methods
- A Class is the blueprint for generating an Object
- Each object is an instance of the class that generated it

```
rect_1 = Rectangle(8, 2)
rect_2 = Rectangle(4, 4)
rect_3 = Rectangle(5, 3)
```

Rectangle is the class

rect_1, rect_2, rect_3 are objects generated by the class Rectangle

```
class Rectangle:

    def __init__(self, width, length):
        self.width = width
        self.length = length

    def area(self):
        return self.width * self.length

    def perimeter(self):
        return 2*self.width + 2*self.length

    def aspect_ratio(self):
        return self.width / self.length

    def diagonal(self):
        return (self.width**2 + self.length**2)**0.5

    def is_square(self):
        return self.width == self.length

    def compare_area(self, other):
        if self.area() > other.area():
            return "The first rectangle is larger"
        elif self.area() < other.area():
            return "The second rectangle is larger"
        else:
            return "Both rectangles have the same area"
```

Classes and Objects

```
rect_1 = Rectangle(8, 2)
rect_2 = Rectangle(4, 4)
rect_3 = Rectangle(5, 3)
```

```
rect_1.area()
```

16

```
print(f"{rect_3.area()}, {rect_3.perimeter()}, {rect_3.aspect_ratio():.2f}, {rect_3.diagonal():.2f}")
```

15, 16, 1.67, 5.83

```
rect_1.is_square()
```

False

```
rect_2.is_square()
```

True

```
rect_1.compare_area(rect_2)
```

'Both rectangles have the same area'

```
rect_1.compare_area(rect_3)
```

'The first rectangle is larger'

Course: Hands-On Data Analytics with Python

Eli Zamir

Session 3: Core Python programming (part 2)

Hands-on session (handout provided separately)